Gradually copy the code from the tutorial into your own jupyter notebook, run and debug until you successfully replicate the tutorial. Do not try to copy and run it all at once, it's not going to work! You need to debug as you go. Use markdown to break the notebook into sections, as in the tutorial. Finish your notebook by answering the following questions, using a combination of text and code directly in the notebook. (Terms like sources and fluxes will gradually become clear as you progress through the tutorial. You are not expected to fully understand all the details.)

1. How many sources (stars) do you find in the data? Histogram their fluxes.
2. What are the mean, median, and standard deviation of the distribution of fluxes.
3. What is the largest outlier in the distribution; how many standard deviations is it away from the mean?

In [1]:
```
import numpy as np
import sep
```

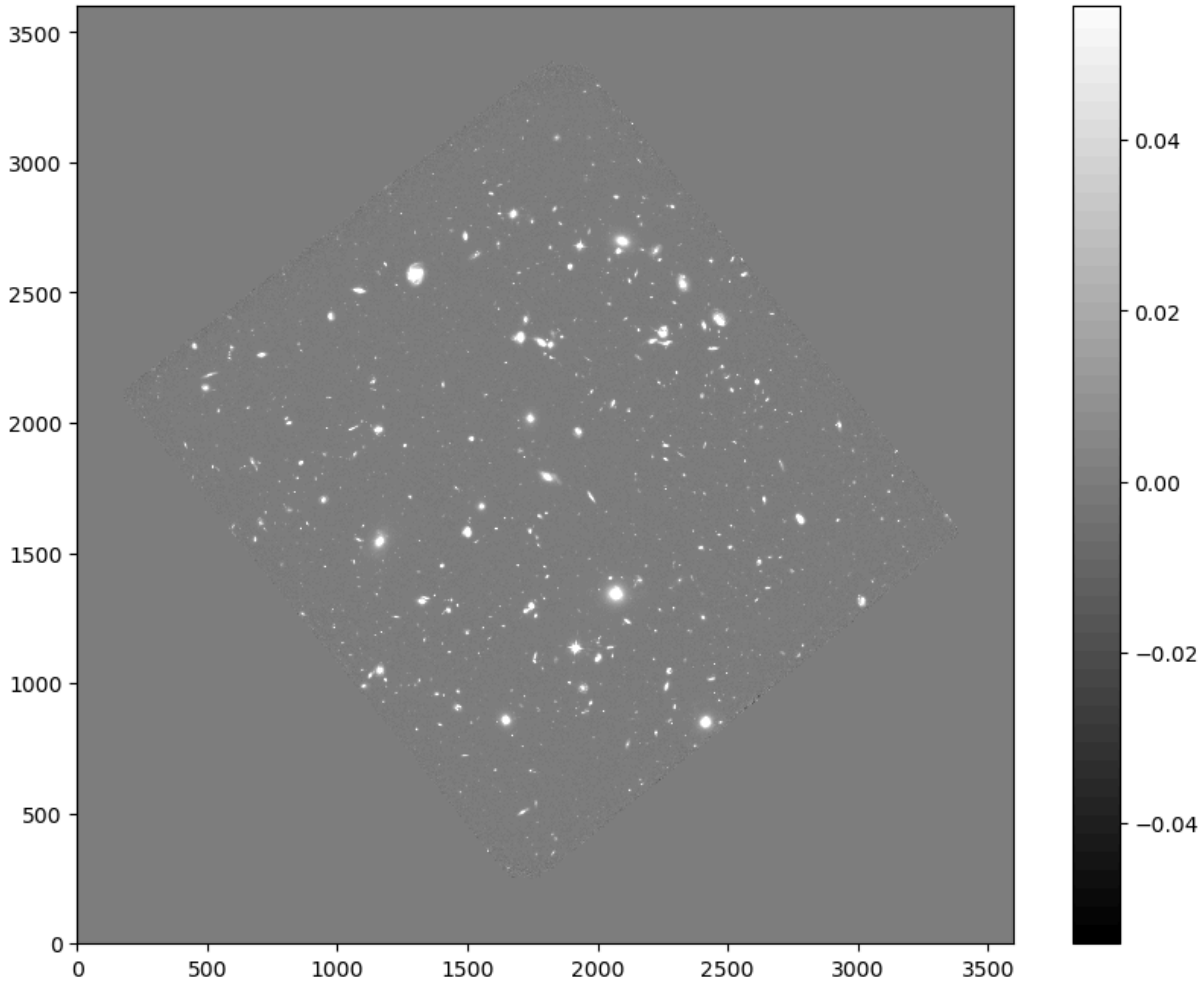
additional setup for reading the test image and displaying plots

In [2]:
```
import fitsio
import matplotlib.pyplot as plt
from matplotlib import rcParams

%matplotlib inline

rcParams['figure.figsize'] = [10., 8.]
```

read image into standard 2-d numpy array

In [3]:
```
data = fitsio.read("data.fits")
```

In [4]:
```
m, s = np.mean(data), np.std(data)
plt.imshow(data, interpolation='nearest', cmap='gray', vmin=m-s, vmax=m+s, c
plt.colorbar();
```



measure a spatially varying background on the image

In [5]:
```
bkg = sep.Background(data)
```

get a "global" mean and noise of the image background:

In [6]:
```
print(bkg.globalback)
print(bkg.globalrms)
```

```
0.0
0.0005398219218477607
```

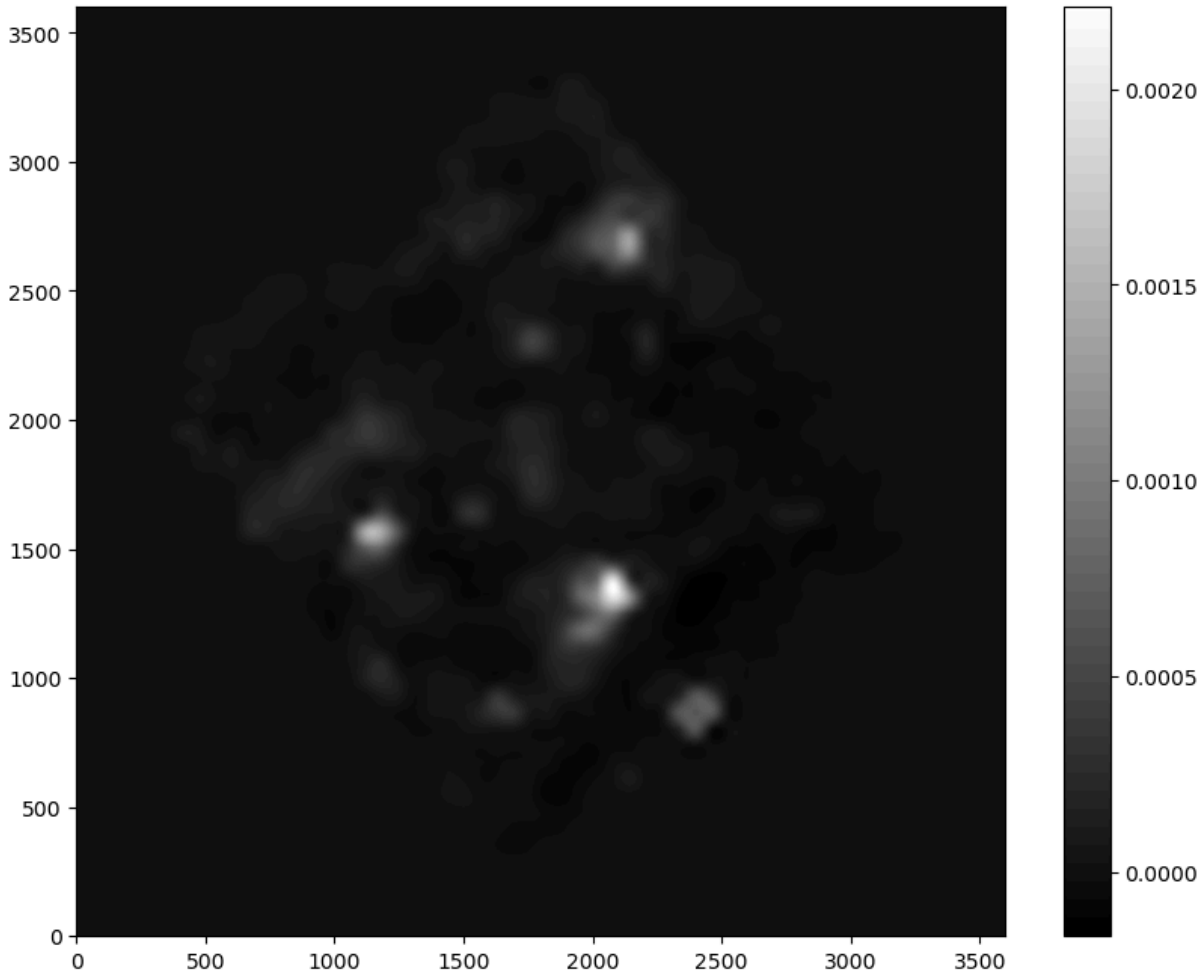
evaluate background as 2-d array, same size as original image

In [7]:
```
bkg_image = bkg.back()
```

bkg_image = np.array(bkg) # equivalent to above

show the background

In [8]:
```
plt.imshow(bkg_image, interpolation='nearest', cmap='gray', origin='lower')
plt.colorbar();
```

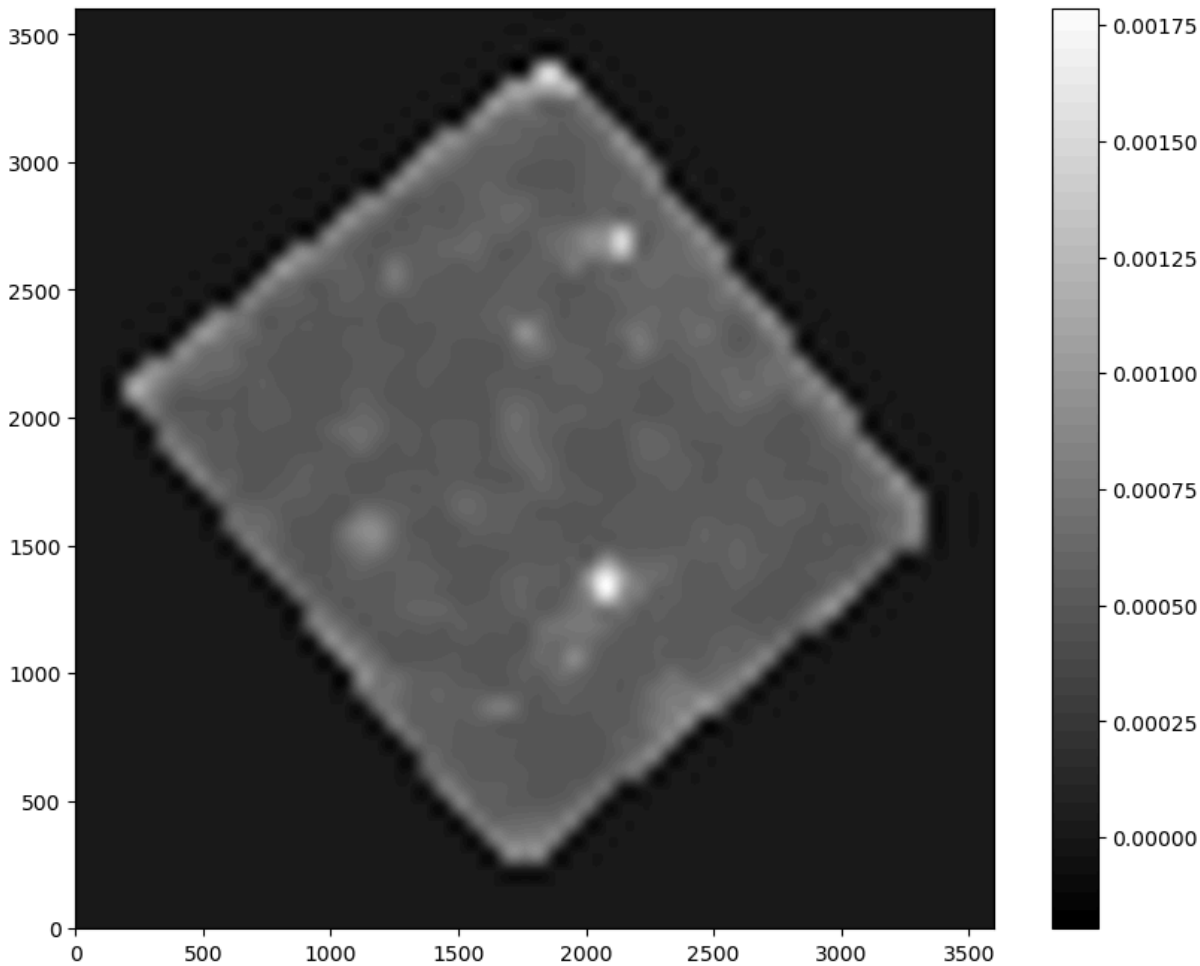


evaluate the background noise as 2-d array, same size as original image

In [9]:
```
bkg_rms = bkg.rms()
```

evaluate the background noise as 2-d array, same size as original image

In [10]:
```
plt.imshow(bkg_rms, interpolation='nearest', cmap='gray', origin='lower')
plt.colorbar();
```



subtract the background

In [11]:
```
data_sub = data - bkg
```

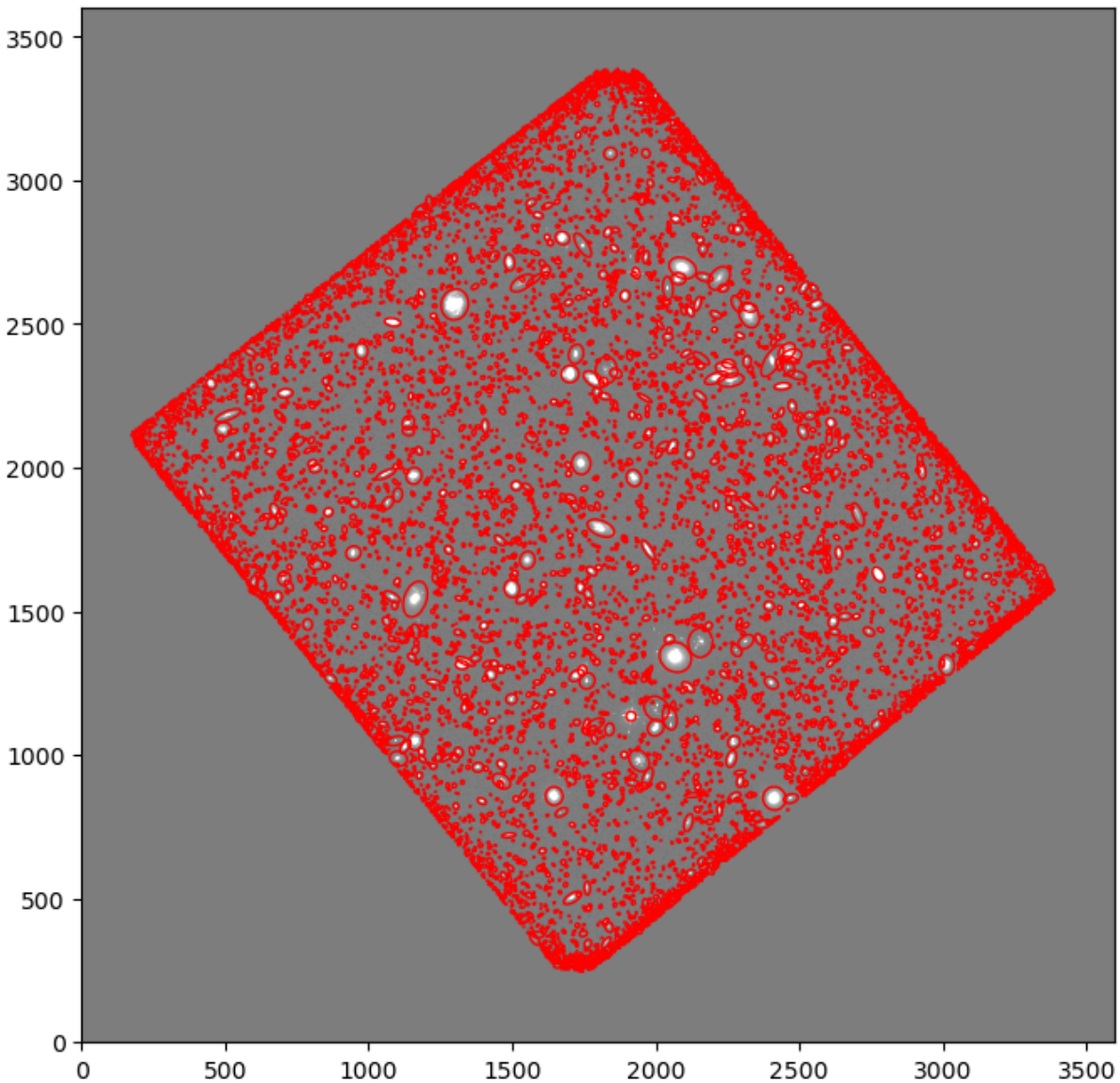
In [12]:
```
objects = sep.extract(data_sub, 1.5, err=bkg.globalrms)
```

In [13]:
```
len(objects)
```

Out[13]: 8640

plot background-subtracted image and plot an ellipse for each object

In [14]:
```
from matplotlib.patches import Ellipse

# plot background-subtracted image
fig, ax = plt.subplots()
m, s = np.mean(data_sub), np.std(data_sub)
im = ax.imshow(data_sub, interpolation='nearest', cmap='gray',
               vmin=m-s, vmax=m+s, origin='lower')


for i in range(len(objects)):
    e = Ellipse(xy=(objects['x'][i], objects['y'][i]),
                width=6*objects['a'][i],
                height=6*objects['b'][i],
                angle=objects['theta'][i] * 180. / np.pi)
    e.set_facecolor('none')
    e.set_edgecolor('red')
    ax.add_artist(e)
```



Availible fields

In [15]:
```
objects.dtype.names
```

Out[15]:
```
('thresh',
 'npix',
 'tnpix',
 'xmin',
 'xmax',
 'ymin',
 'ymax',
 'x',
 'y',
 'x2',
 'y2',
 'xy',
 'errx2',
 'erry2',
 'errxy',
 'a',
 'b',
 'theta',
 'cxx',
 'cyy',
 'cxy',
 'cflux',
 'flux',
 'cpeak',
 'peak',
 'xcpeak',
 'ycpeak',
 'xpeak',
 'ypeak',
 'flag')
```

In [16]:
```
flux, fluxerr, flag = sep.sum_circle(data_sub, objects['x'], objects['y'],
                                     3.0, err=bkg.globalrms, gain=1.0)
```

show the first 10 objects results:

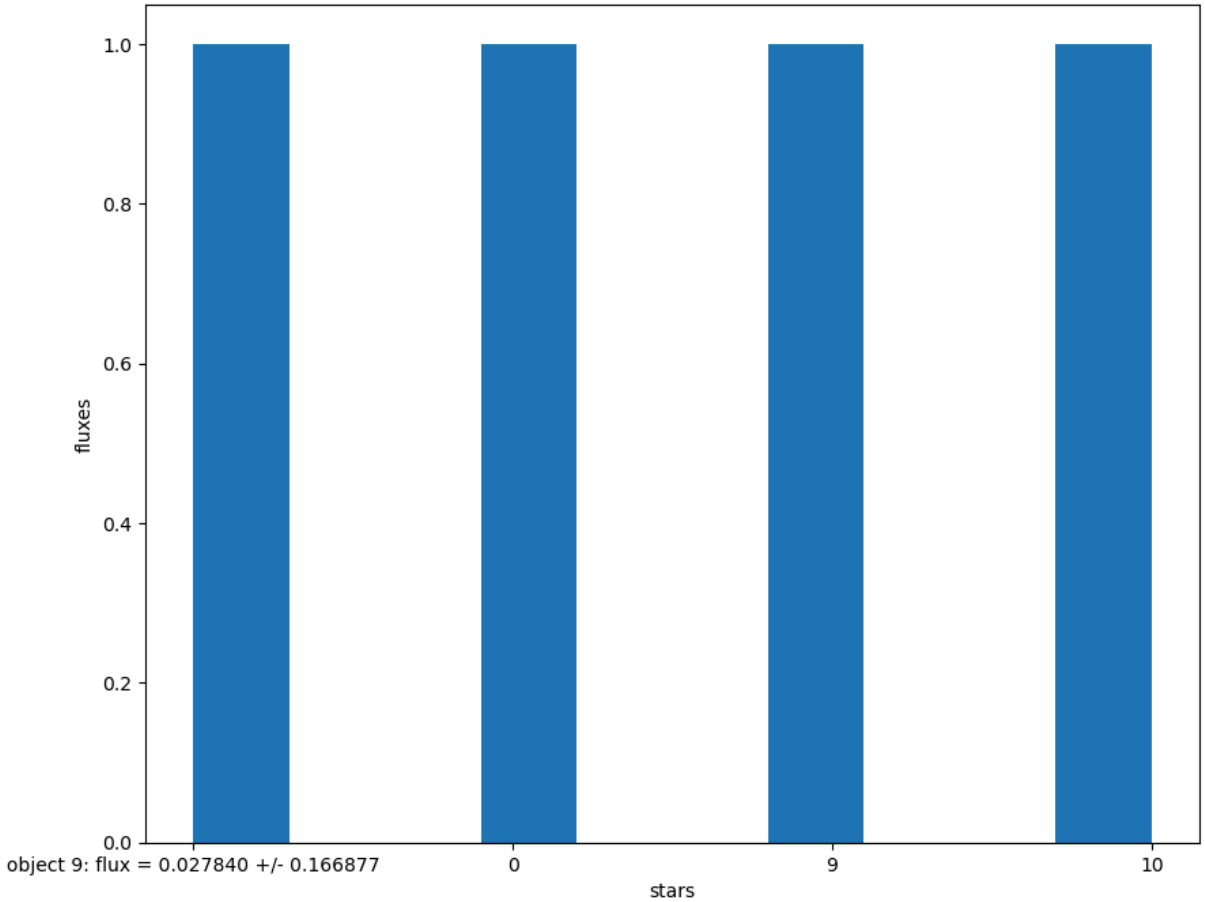
In [17]:
```
for i in range(10):
    print("object {:d}: flux = {:f} +/- {:f}".format(i, flux[i], fluxerr[i]))
```

```
object 0: flux = 0.031282 +/- 0.176890
object 1: flux = 0.031018 +/- 0.176142
object 2: flux = -0.024388 +/- 0.002883
object 3: flux = 0.001947 +/- 0.044219
object 4: flux = 0.012457 +/- 0.111649
object 5: flux = -0.011228 +/- 0.002875
object 6: flux = 0.029368 +/- 0.171394
object 7: flux = -0.009126 +/- 0.002875
object 8: flux = 0.048023 +/- 0.219161
object 9: flux = 0.027840 +/- 0.166877
```

In [18]:
```
x = ("object {:d}: flux = {:f} +/- {:f}".format(i, flux[i], fluxerr[i]),0,9,
y = ("object {:d}: flux = {:f} +/- {:f}".format(i, flux[i], fluxerr[i]))
bins = 10
```

```
plt.hist(x, bins = bins)
plt.xlabel('stars')
plt.ylabel('fluxes')
```

Out[18]: Text(0, 0.5, 'fluxes')



In [ ]:

In [ ]:

In [ ]:

In [ ]: